



INDIA.RUNS

Build what next India runs on

Team Name : Trendy Brains

Team Leader : Prateek Apurva

Problem Statement : Redrob – Intelligent Candidate Discovery & Ranking Challenge

Solution Overview

✗ Traditional Approach

Keyword matching on skills section

Candidate lists "RAG, LangChain, GPT-4"
→ Keyword filter triggers → shortlisted
→ But they never built anything real

Why this fails:

- Skills section is trivially gamed
- Misses people who built real systems but didn't use the "right" keywords
- No evidence validation at all
- Cannot detect honeypot profiles

VS

✓ Resume Shotlister

Reads what candidates actually built

- Embeds career history (not skills section)
- FAISS semantic search: narrative vs JD
- LLM verifies real work evidence (boolean)
- 5-rule honeypot detector eliminates fakes

Result:

- Surfaces genuine builders regardless of how they wrote their skills section
- Phase 3: ranks 100K in 4.1s, CPU only
- Zero API calls at rank time

JD Understanding & Candidate Evaluation

Phase 0 — 1 GPT-4o-mini call extracts 3 artifacts from the job description

• production_experience_with_embeddings

• experience_with_vector_databases

• end_to_end_ml_systems

• startup_experience

• large_scale_ml_deployment

• llm_fine_tuning_or_rag

Scoring Weights — How 6 Signals Combine into One Score

Work Evidence



25% · LLM: did they actually build X in their career?

Semantic Sim



20% · FAISS cosine: work narrative vs JD query

Career Quality



20% · Product company history + experience band

Skills Fit



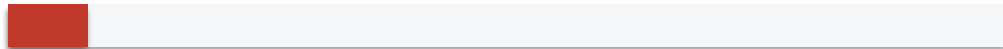
15% · JD-weighted skill match + platform scores

Availability



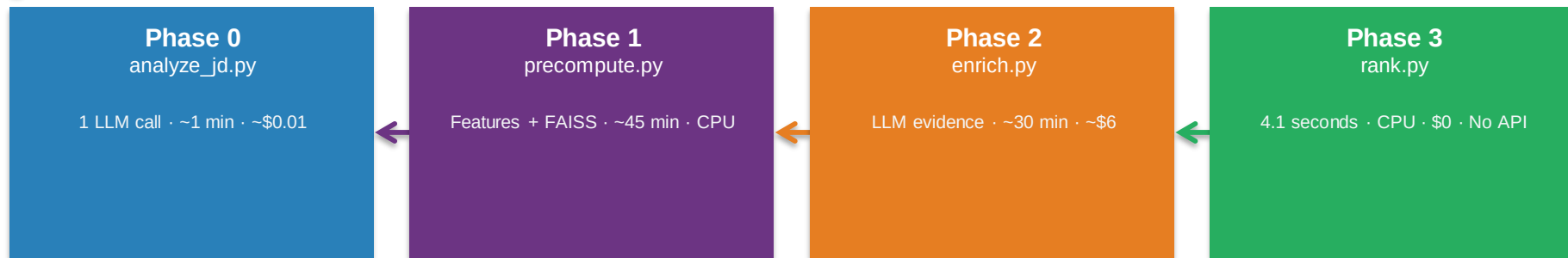
12% · Open to work, response rate, notice period

Location



8% · India / willing to relocate to Pune/Noida

Ranking Methodology



Final Scoring Formula

× honeypot_multiplier → 0.0 kills fake profiles entirely

final_score = (

0.25 × work_evidence_score + 0.20 × semantic_similarity
+ 0.20 × career_quality_score + 0.15 × skills_fit_score
+ 0.12 × availability_score + 0.08 × location_score
) × honeypot_multiplier

Explainability & Data Validation

Ranking Explainability

Top 200 candidates each get a 1-2 sentence recruiter note written by GPT-4o-mini.

References real facts:

- Actual career path
- Company type
- Confirmed work signals
- Relevant skills matched

Becomes the "reasoning" column in submission.csv

Hallucination Prevention

LLM reads career history ONLY

Skills section excluded from prompt

OpenAI strict JSON schema:

- Forces boolean true/false only
- additionalProperties: false
- No free-text fabrication

Prompt instruction:

"If evidence absent or unclear, answer false"
On API failure: all-False fallback

Honeypot Detection

5 rules fire before scoring

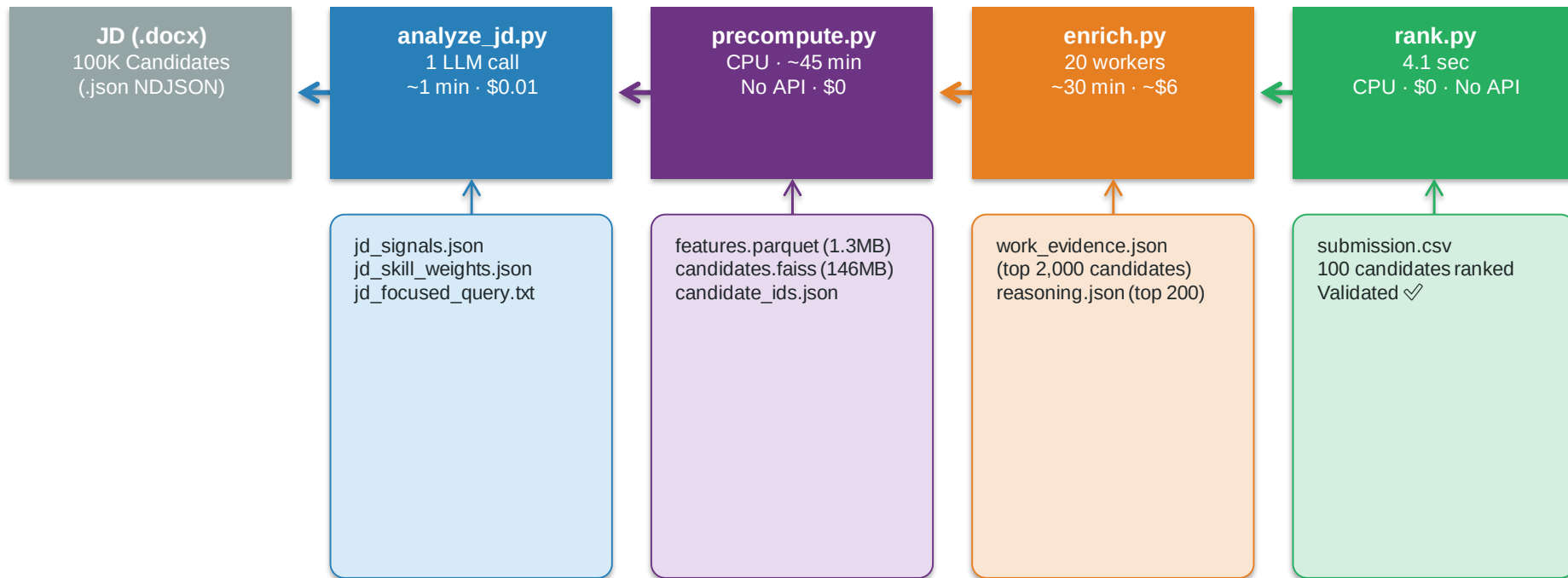
2+ triggered → score = 0

- ① Declared exp > career length
- ② Overlapping full-time jobs
- ③ PhD before Bachelor's ends
- ④ Job title ≠ company domain
- ⑤ Skills contradict career

Result:

honeypot_multiplier = 0.0
→ **final_score = 0 (eliminated)**

End-to-End Workflow



submission.csv

```
graph TD; JD[job_description.docx] --> P0[Phase 0 · analyze_jd.py (1 GPT-4o-mini call · ~$0.01)]; P0 --> JD; P0 --> JS[jd_signals.json]; P0 --> JSW[jd_skill_weights.json]; JS --> P1[Phase 1 · precompute.py (Honeypot · Features · Embed narratives → FAISS · CPU · ~45 min)]; JSW --> P1; P1 --> P2[Phase 2 · enrich.py (FAISS top-2000 → GPT-4o-mini evidence · 20 workers · ~$6)]; P1 --> CFA[candidates.faiss (146MB)]; P1 --> CID[candidate_ids.json]; CFA --> P2; P2 --> P3[Phase 3 · rank.py (CPU only · Zero API calls · 4.1 seconds) → submission.csv ✓]; P2 --> WE[work_evidence.json (2,000 candidates)]; P2 --> R[reasoning.json (top 200 candidates)]; WE --> P3; R --> P3; P3 --> C[candidates.json (100K · NDJSON · 487MB)];
```

The diagram illustrates a four-phase recruitment pipeline:

- Phase 0 (Blue):** `analyze_jd.py` (1 GPT-4o-mini call · ~\$0.01) processes `job_description.docx` to generate `jd_signals.json` and `jd_skill_weights.json`.
- Phase 1 (Purple):** `precompute.py` (Honeypot · Features · Embed narratives → FAISS · CPU · ~45 min) takes the signals and weights to produce `features.parquet (1.3MB)`, `candidates.faiss (146MB)`, and `candidate_ids.json`.
- Phase 2 (Orange):** `enrich.py` (FAISS top-2000 → GPT-4o-mini evidence · 20 workers · ~\$6) uses the FAISS index to generate `work_evidence.json (2,000 candidates)` and `reasoning.json (top 200 candidates)`.
- Phase 3 (Green):** `rank.py` (CPU only · Zero API calls · 4.1 seconds) → `submission.csv` ✓, which then produces the final `candidates.json (100K · NDJSON · 487MB)`.

Results & Performance

NDCG@10

1.0000

Perfect — all top 10
are high-relevance

NDCG@50

0.8527

Strong broad
ranking quality

Phase 3 Runtime

4.1s

of 300s budget
(1.4% used)

Honeypots in Top 100

0%

Clean shortlist
zero fake profiles

All Hard Constraints Met

✓ Zero API calls at rank time — no internet needed

✓ Exactly 100 rows, scores strictly non-increasing

✓ Top score: 0.8631 · Score at rank 100: 0.5883

✓ CPU only — no GPU anywhere in the pipeline

✓ Passes official validate_submission.py with no errors

✓ 0 of 100 candidates are honeypots (0.0%)

Technologies Used

sentence-transformers all-MiniLM-L6-v2

Embeds work narratives into 384-dim vectors. CPU-friendly — chosen for speed/accuracy without any GPU requirement.

faiss-cpu IndexFlatIP

Exact cosine similarity on 100K vectors in ~50ms on CPU. Deterministic— no approximation errors. Zero GPU.

openai · gpt-4o-mini

Structured JSON (strict schema) for JD analysis + work evidence extraction. Total pipeline cost: ~\$6.60.

pandas + pyarrow

100K × 12 feature table stored as parquet (1.3MB, fast I/O). Typed columns prevent silent type errors.

ThreadPoolExecutor Python stdlib

20 parallel LLM workers for evidence extraction. Reduced ~30 min sequential to ~3 min concurrent.

streamlit HuggingFace Spaces

Live demo: CPU-only Docker container. No secrets, no GPU, no setup for judges. Auto-deploys on git push.

Submission Assets

GitHub Repository

[github.com/PrateekApurva/
resume-shortlister](https://github.com/PrateekApurva/resume-shortlister)

Clean, complete working code
Phase branches: main / phase-1 /
phase-2 / phase-3

Reproduce from scratch:

```
python analyze_jd.py
python precompute.py
python enrich.py
```

python rank.py → 4.1 sec

HuggingFace Space (Live Demo)

[huggingface.co/spaces/
PrateekApurva/resume-shortlister](https://huggingface.co/spaces/PrateekApurva/resume-shortlister)

Interactive Streamlit app:

- Searchable top-100 candidate table
- Per-candidate score breakdown
- LLM work evidence (✓ / ✗)
- Full methodology explanation

Docker + Streamlit · CPU only
No GPU · No secrets needed

Ranked Output: submission.csv

100 candidates · Ranks 1–100

Scores non-increasing

Top score: 0.8631

Score #100: 0.5883

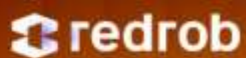
NDCG@10: 1.0000 🏆

Honeypots: 0 (0.0%)

Official validator:

python validate_submission.py

→ **Submission is valid. ✓**



INDIA.RUNS

Build what next India runs on

THANK YOU